



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

4 - “User Authentication” [Chapter 3]

Gabriele D'Angelo <g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>

Outline



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Means** of Authentication
- **Password**-based Authentication
- **Token**-based Authentication
- **Biometric** Authentication
- **Remote** User Authentication
- **Security Issues** for User Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Authentication

The system does not take your word for it (your claim) but requires proof. The verification process serves to link the entity making the claim to the identity it has declared, usually through something the entity knows (a password), has (a token), or is (biometrics).

An entity can be: a human user attempting to log in, a software process or microservice that needs to communicate with another server, or a hardware device (such as an IoT sensor) that connects to a network.

Authentication never starts from scratch; it always begins with a claim. When you enter a username (e.g., `mario.rossi`), you are telling the system: “Hey, I affirm that I am Mario Rossi.” Identity can be declared by the entity itself (“by”) or by another system on its behalf (“for”), as is the case, for example, in Identity Provider protocols (OAuth).



Source: www.veto.net



Authentication Process: the process of verifying an entity's identity

Authentication Process



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Fundamental building block and **primary line of defense**

of the entire cybersecurity system

- Basis for **access control** and **user accountability**

- Access Control: The system cannot determine what you are permitted to do (Authorization) unless it first knows with absolute certainty who you are (Authentication).
- User Accountability: In the event of anomalies or attacks, you must be able to trace a specific action back to a specific user. Without strong authentication, actions become anonymous and can be repudiated.

- Typically composed of **two separate steps**:

The user provides an identifier to the system.
Examples: Enter your username or your email address.
At this stage, the system only knows that someone is claiming the identity associated with that identifier.

① **identification step**: “presenting an **identifier** to the security system”

② **verification step**: “**presenting** or **generating** authentication information

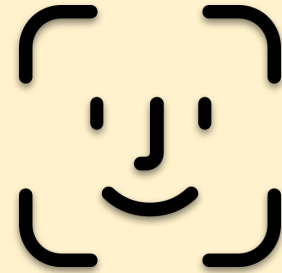
that ~~corroborates~~ ^{confirms} the ^{link} **binding** between the **entity** and the **identifier**”

This is the process of validating the link (binding) between that identifier and the actual entity. The user presents or generates proof (the “authentication information”). The system verifies that this specific proof is mathematically or logically linked to the identifier provided in step 1. Examples: Entering a password (something you know). Generating an OTP (One-Time Password) using a cryptographic algorithm on your smartphone (something you have) based on a shared secret key. If the verification is successful, the binding is confirmed and the system considers you authenticated.

Authentication Process

On your personal phone, the identification step is automatic (the phone already knows it belongs to a single owner). As a result, your face is your password (the verification factor). It works exactly like a PIN: the phone scans your face, converts it into a mathematical value, and compares it to the one stored in its memory. If they match, the "password" is correct and the phone unlocks.

- Fundamental building block and **primary**
- Basis for **access control** and **user authentication**
- Typically composed of two steps:
 - **identification step**: "presenting an **identifier** to the security system"
 - **verification step**: "**presenting** or **generating** authentication information that corroborates the **binding** between the **entity** and the **identifier**"



Face ID: is your face your
identifier or your
password?

This slide introduces the four pillars of authentication, that are the categories of information a system may require to verify a user's identity. In technical terms, these are called authentication factors.

User Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

methods for
the ~~four pillars~~ authenticating user identity
are based on:

- 1 something the individual **knows**
 - password, PIN, answers to pre-arranged questions
- 2 something the individual **possesses** (token)
 - smartcard, electronic keycard, physical key
- 3 something the individual **is** (static biometrics)
 - fingerprint, retina, face
- 4 something the individual **does** (dynamic biometrics)
 - voice pattern, handwriting, typing rhythm

User Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

the **four means**

entity

The password recovery questions

(i.e., security questions):

they must be as secure as the

password

something the
individual **knows**

something the
individual **possesses**

something the
individual **does**

- password, PIN,
answers to
pre-arranged
questions

- smart card,
electronic
keycard,
physical key

biometrics)

- fingerprint,
retina, face

(dynamic
biometrics)

- voice pattern,
handwriting,
typing rhythm

User Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

the **four means** of authenticating user identity are based on:

something the individual **knows**

- *password, PIN, answers to pre-arranged questions*

something the individual **possesses** (token)

- *smartcard, electronic keycard, physical key*

something the individual **is** (static biometrics)

- *fingerprint, retina, face*

something the individual **does** (dynamic biometrics)

- *voice pattern, handwriting, typing rhythm*



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Passwords

Password Authentication

1. Privileges (System-Wide Capabilities)

- Definition: A user's general capabilities and roles within the system, independent of any specific object.

- Core Question: "What types of actions is this user authorized to perform?"

- What they are: Roles, profiles, or macro-permissions (e.g., Reader, Writer, Administrator).

- Example: User ID mario.rossi has "Write" privileges and belongs to the Human Resources group.

2. Access Control (Resource-Specific Verification)

- Definition: The practical enforcement process performed every time a user attempts an action on a specific resource (file, database table, web page).

- Core Question: "Does this specific User ID have the right to perform this specific action on this specific object?"

- What it is: The intersection of the user's privileges and the resource's security policy (e.g., Access Control Lists - ACLs).

- Example: User mario.rossi tries to open `stipendi_manager.pdf`. The system checks the file's rules: "Access restricted to Executives only". Since Mario is in HR, the system enforces ACCESS DENIED.

- Widely used line of defense against intruders
 - user provides name/**login** (i.e., user ID) and **password**
 - system **compares password** with the **one stored for that specified**

login

Modern systems do not store passwords in plain text in the database; instead, they store an encrypted version called a hash. The system then compares the hashes, not the actual passwords.

The first check is binary: Does this user have an active account? Are they authorized to log in? If the User ID is disabled (for example, an employee who has left the company), access is denied even if the password is correct.

The **user ID**: 1) **determines that the user is authorized to access the**

system; 2) **determines the user's privileges**; 3) **is used in access control**

Once you've logged in, the system needs to know what you're allowed to do. The User ID is the key that tells the system what your role is.

This is directly related to the previous point, but at a granular level (for individual files or resources). When you try to open a file, the system asks: "Does the User ID 'student_123' have permission to edit this specific file?" Your User ID tracks every single action you take within the system to verify your permissions in real time.

Password Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

... in the proper way...
(see later)

- Widely used line of defense
 - user provides name/**login** (i.e., user ID) and **password**
 - system **compares** password with the **one stored** for that specified login
- The **user ID**: 1) determines that the user is **authorized to access the system**; 2) determines the user's **privileges**; 3) is used in **access control**

Password Vulnerabilities



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

offline
dictionary
attack

popular
password
attack

workstation
hijacking

exploiting
multiple
password
use

specific
account
attack

password
guessing
against
single user

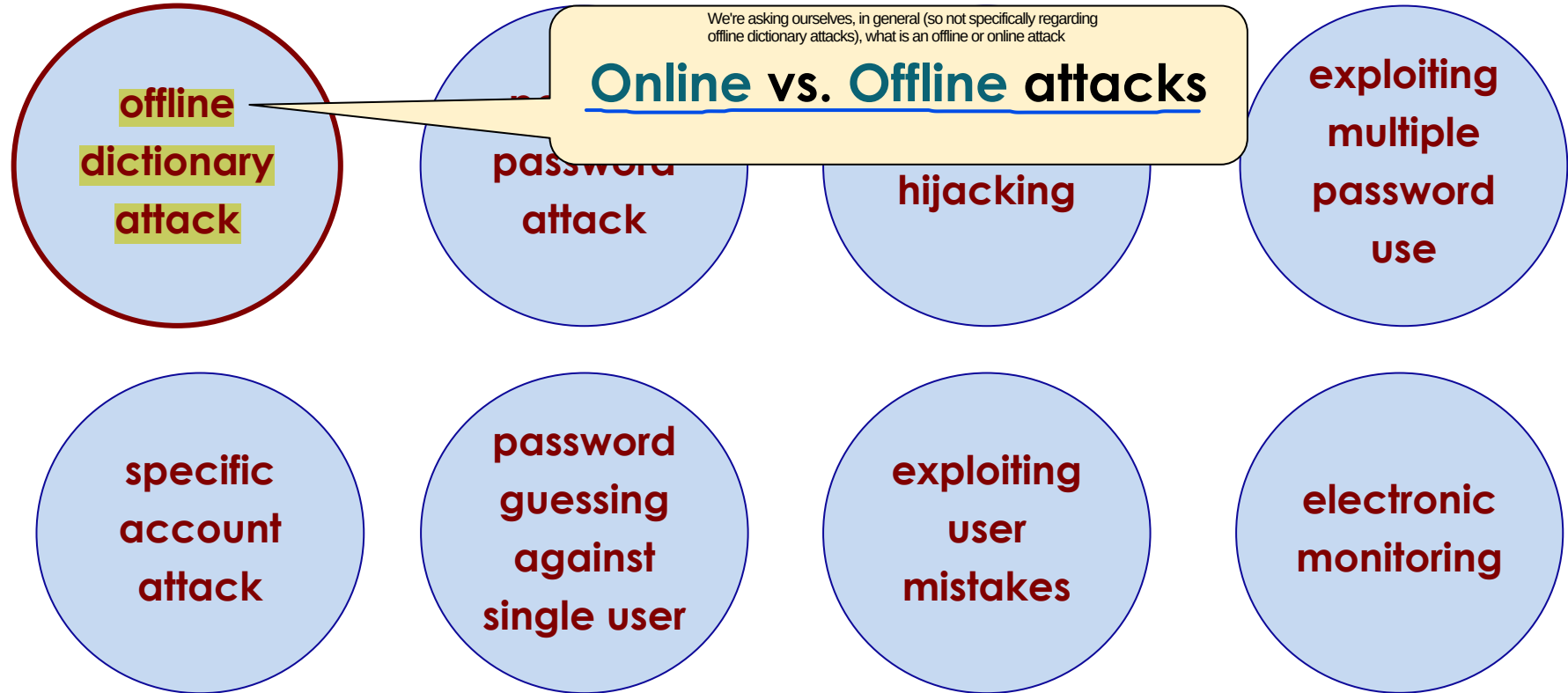
exploiting
user
mistakes

electronic
monitoring

Password Vulnerabilities



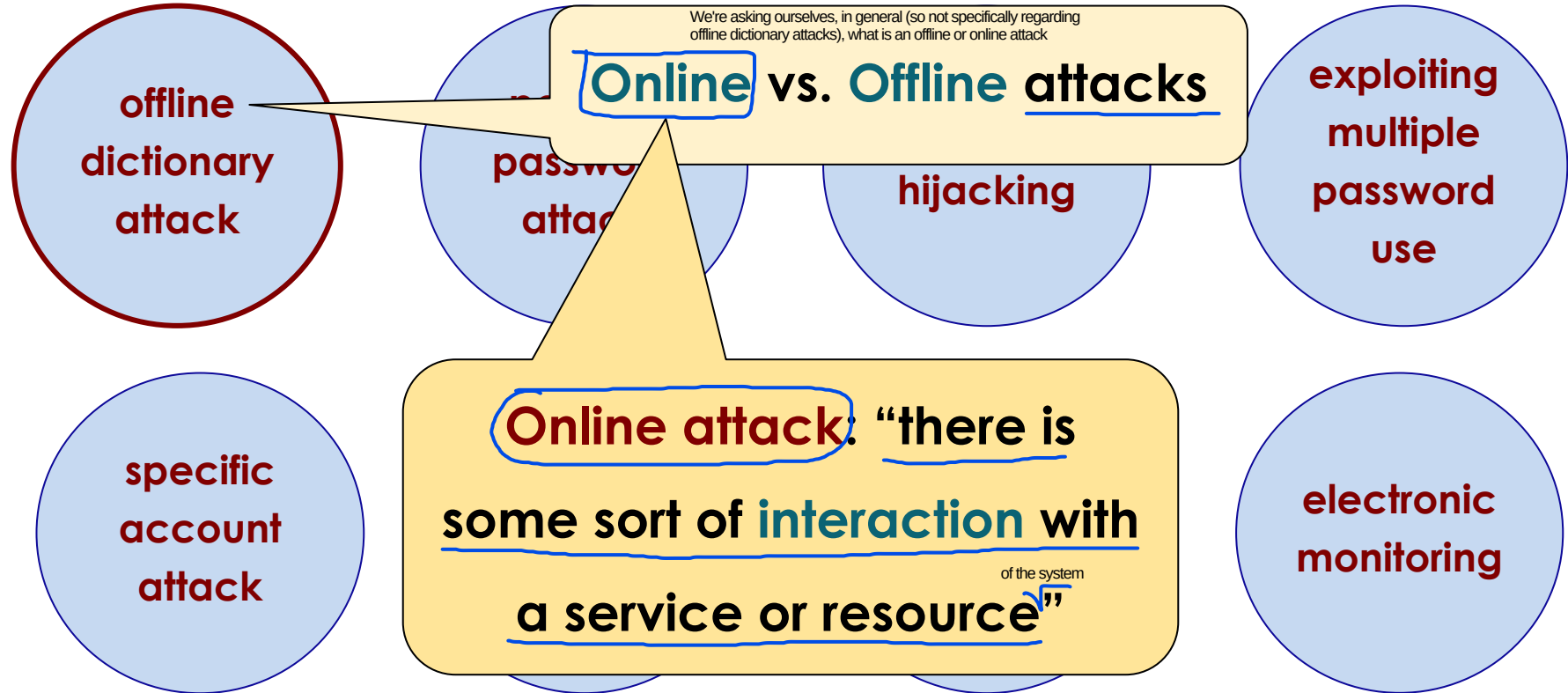
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Password Vulnerabilities



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Password Vulnerabilities



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Online vs. Offline attacks

offline
dictionary
attack

password
attack

hijacking

exploiting
multiple
password
use

specific
account
attack

Offline attack: “no direct
with the system
interaction. Usually involves
intercepted (or stolen) data”

electronic
monitoring

Password Vulnerabilities



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

offline
dictionary
attack

Example: the attacker has been
able to get a copy of the database
with **encrypted passwords**

specific
account
attack

password
guessing
against
single user

exploiting
user
mistakes

electronic
monitoring

Password Vulnerabilities



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

offline
dictionary
attack

Example: the attacker has been
able to get a copy of the database
with encrypted passwords

specific
account
attack

He/She will try to find the passwords
comparing them with “common words”
that can be found in a dictionary

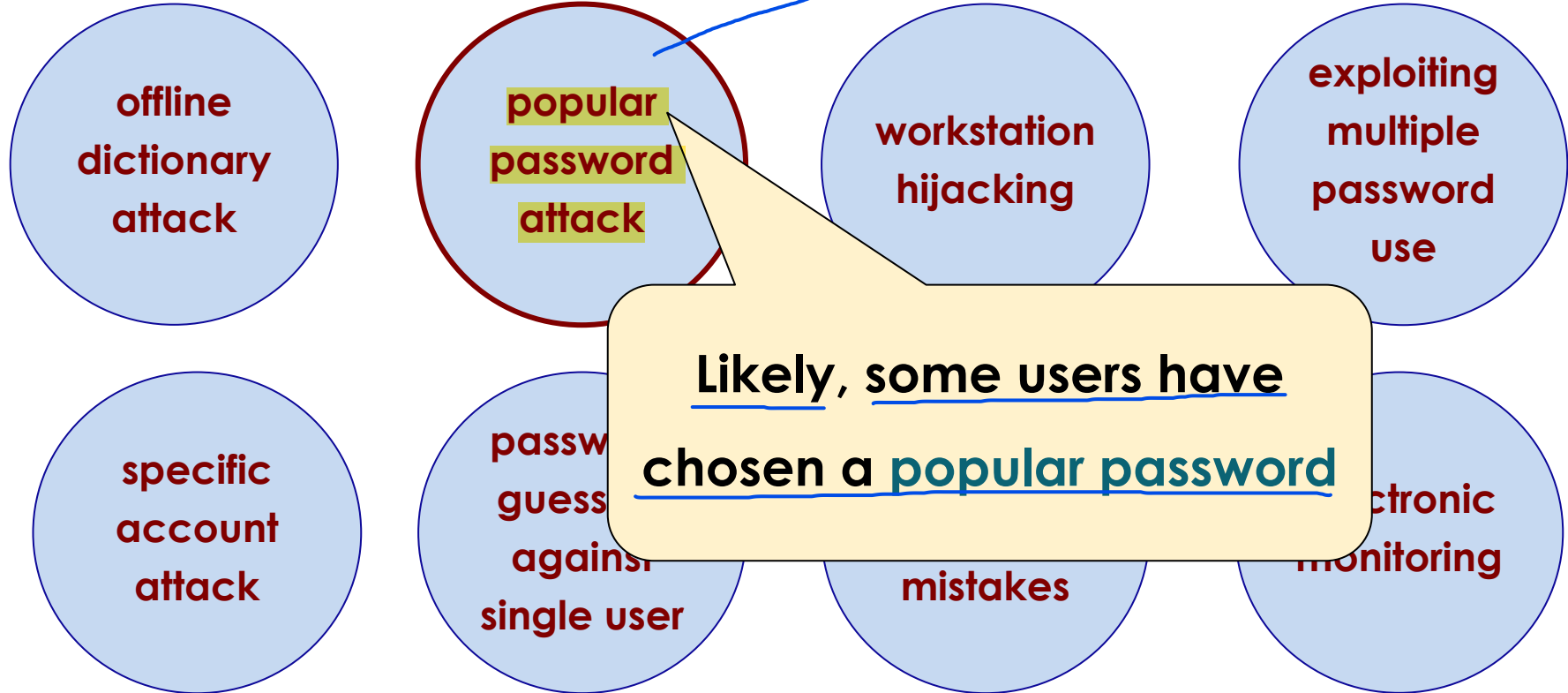
Password Vulnerabilities

In a popular password attack, the attacker:

- Compiles a list of thousands of different usernames or email addresses (often obtained from past public data breaches).
- Selects a single, extremely common password (the "popular password," such as 'Password123' or '123456').
- Tries that single password on every account in the list, one by one.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Password Vulnerabilities

This occurs when authentication is based on passwords, but a user leaves their device unlocked and unsupervised (e.g., during a restroom or lunch break). An attacker can then step in and use the active session, completely bypassing the security limitations imposed by password authentication.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**offline
dictionary
attack**

**popular
password
attack**

**workstation
hijacking**

**exploiting
multiple
password
use**

**specific
account
attack**

**password
guessing
against
single user**

**exploiting
user
mistakes**

**electronic
monitoring**

Password Vulnerabilities

Exploiting multiple password use relies on users' bad habit of using the same email and password combination on many different sites. Here's how the attack works:

- The theft: The attacker breaches an insecure site (e.g., a small forum) and steals the database containing user credentials.
- Exploitation: Using automated software, the attacker tries the same stolen credentials on major, well-protected sites (banks, email, social media, corporate networks).



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**offline
dictionary
attack**

**popular
password
attack**

**workstation
hijacking**

**exploiting
multiple
password
use**

**specific
account
attack**

**password
guessing
against
single user**

**exploiting
user
mistakes**

**electronic
monitoring**

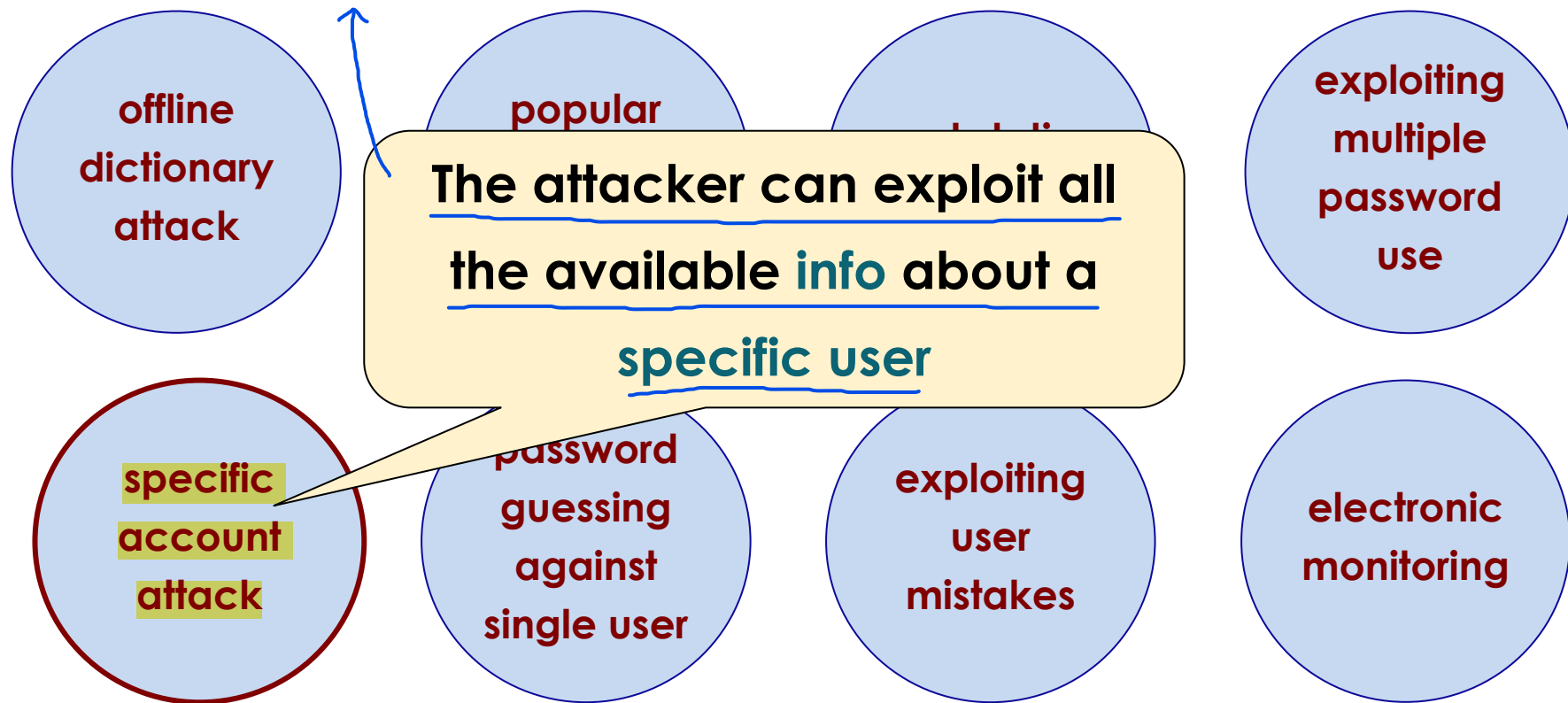
Before attempting to log in, the attacker gathers information about the victim using public sources and social media (a process known as OSINT - Open Source Intelligence). Once the attacker has created a "profile" of the victim, they use this information in two main ways: creating a custom dictionary or bypassing security questions for password resets. Unlike a Popular Password Attack (where a common password is tried on many users), here the effort is focused 100% on a single person. The attack is more effective the more distracted the victim is regarding their privacy on social networks.

Password Vulnerabilities

The attacker does not target just any user, but has a specific, predetermined target. The key feature of this attack is that the attacker gathers and exploits all available personal information about the victim to guess their password.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



This type of attack has a major weakness if carried out directly on the website's login page (i.e., as an Online Attack): it is extremely easy to detect and block. Since all attempts are directed at the exact same account, modern security systems detect them immediately. After 3, 5, or 10 consecutive failed attempts, the server activates countermeasures:

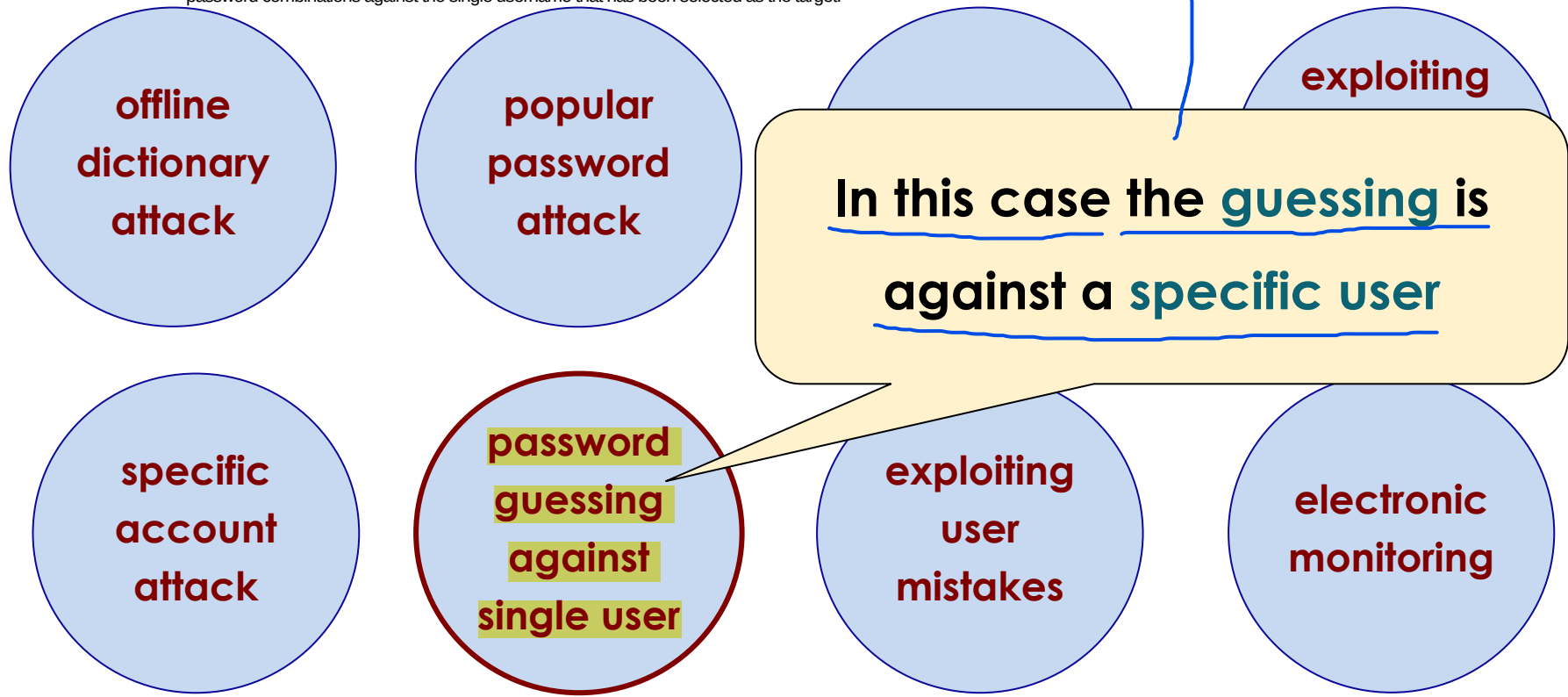
- Account Lockout: Temporarily locks the account (e.g., for 30 minutes), preventing the attacker from making further attempts.
- IP Blocking / Rate Limiting: Detects that an external computer is sending too many requests for that user and blocks its IP address.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Password Vulnerabilities

- Specific Account Attack: This is a larger concept. It includes not only attempting to guess the password, but also the entire process of gathering information about the victim.
- Password Guessing against a Single User: This is the specific action of testing various password combinations against the single username that has been selected as the target.



Password Vulnerabilities



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**offline
dictionary
attack**

**popular
password
attack**

**workstation
hijacking**

**exploiting
multiple
password
use**

**specific
account
attack**

**password
guessing
against
single user**

**exploiting
user
mistakes**

**electronic
monitoring**

Password Vulnerabilities

Electronic Monitoring refers to a category of attacks in which the attacker does not guess the password or steal a database, but secretly intercepts the user's actions in real time as the user types or enters their credentials. It is a passive, sniffing/eavesdropping, that can occur at both the hardware and software levels.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**offline
dictionary
attack**

**popular
password
attack**

**workstation
hijacking**

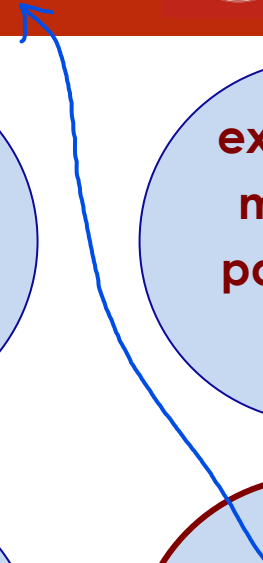
**exploiting
multiple
password
use**

**specific
account
attack**

**password
guessing
against
single user**

**exploiting
user
mistakes**

**electronic
monitoring**



Countermeasures



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- 1 • Strict access controls to prevent anyone from stealing the password (or hash) file.
If the attacker cannot download this file, they cannot launch an offline attack.
Controls to prevent unauthorized access to password file
- 2 • **Intrusion detection** measures
- 3 • **Rapid reissuance** of **compromised passwords** → If the system detects that a user's credentials have been compromised (for example, by finding them in a public data leak on the web), it immediately disables the old password and requires the user to generate a new one on the fly.
reset
- 4 • **Account lockout mechanisms**
It is the ultimate defense against password guessing for a single user. After 3 or 5 failed attempts, the account is locked, rendering online brute-force attacks ineffective.
- 5 • **Policies** to ^{avoid} ~~inhibit~~ users from selecting **common passwords**
- 6 • **Training** ~~in~~ and **enforcement** of **password policies**
- 7 • Automatic **workstation logout**
- 8 • **Policies** against **similar passwords** on **network devices**
Specific rules prohibiting the use of identical or very similar passwords (e.g., changing only the final year, such as Winter2025! and Winter2026!) on different devices within the corporate network. This way, if a hacker exploits the use of multiple passwords on one device, they will not be able to use similar passwords to compromise the rest of the network.

Countermeasures



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The account is **locked** after a
given number of **failed attempts**

- Controls to prevent **unauthorized access**
- **Intrusion detection** mechanisms
- **Rapid reissuance** of compromised passwords
- Account **lockout mechanisms**
- **Policies** to inhibit users from selecting **common passwords**
- **Training** in and **enforcement** of **password policies**
- Automatic **workstation logout**
- **Policies** against **similar passwords** on **network devices**

Countermeasures



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Controls to prevent **unauthorized access**
- **Intrusion detection** mechanisms
- **Rapid reissuance** of compromised passwords
- **Account lockout mechanisms**
- **Policies** to inhibit users from setting weak passwords
- **Training** in and **enforcement** of security policies
- Automatic **workstation logouts**
- **Policies** against **similar passwords**

The account is **locked** after a given number of **failed attempts**

It is useful but it is also dangerous... **abused** (i.e., **Denial of Service**)

Countermeasures



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Controls to prevent **unauthorized access** to **password file**
- **Intrusion detection** measures
- **Rapid reissuance** of **compromised passwords**
- Account **lockout mechanisms**
- **Policies** to inhibit users from selecting **common passwords**
- **Training** in and **enforcement** of **password policies**
- Automatic **workstation logout**
- **Policies** against **similar passwords** on **network devices**

Password Storage



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Storing the passwords “**in clear**” is a **very bad idea** (and... *passwords delivered by email?*)
- The **sysadmin** would be able to **access all users' passwords** (*and to check if they're used in other systems*)
- An **intruder** would get a **very valuable asset**
- A widely used **password security technique** is the use of **hashed password** and a **salt value**
- This means that the **passwords are never stored “in clear”**



Salt

Load

Password file

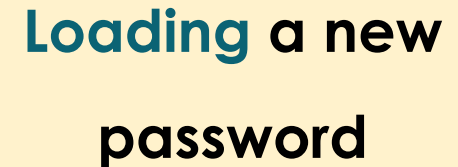
User ID	Salt	Hash code
---------	------	-----------

[illegible]

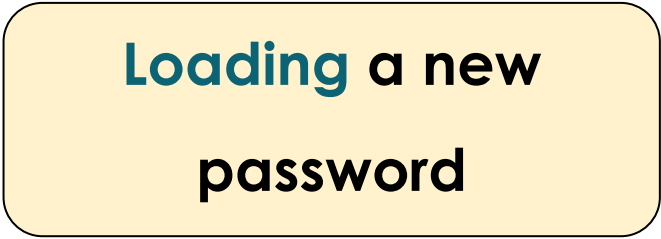
When a user registers or changes their password, the system calculates the hash before saving it to the database. If we used a super-fast hash function, a modern computer could calculate billions of hashes per second. If a hacker stole the database, they could crack short or common passwords in just a few minutes. By using a Slow Hash Function, the system intentionally introduces a computational “delay”. The user signing up will never notice a fraction of a second of delay, but for an attacker who has stolen the database and must try billions of combinations, that 100ms delay multiplied by each attempt completely destroys the feasibility of the attack (it would take far too long).

Loading a new password

Each password is
obtained through
a different salt value



The salt values are chosen at random and stored (in clear) in the password file



The salt values are chosen at random and stored (in clear) in the password file

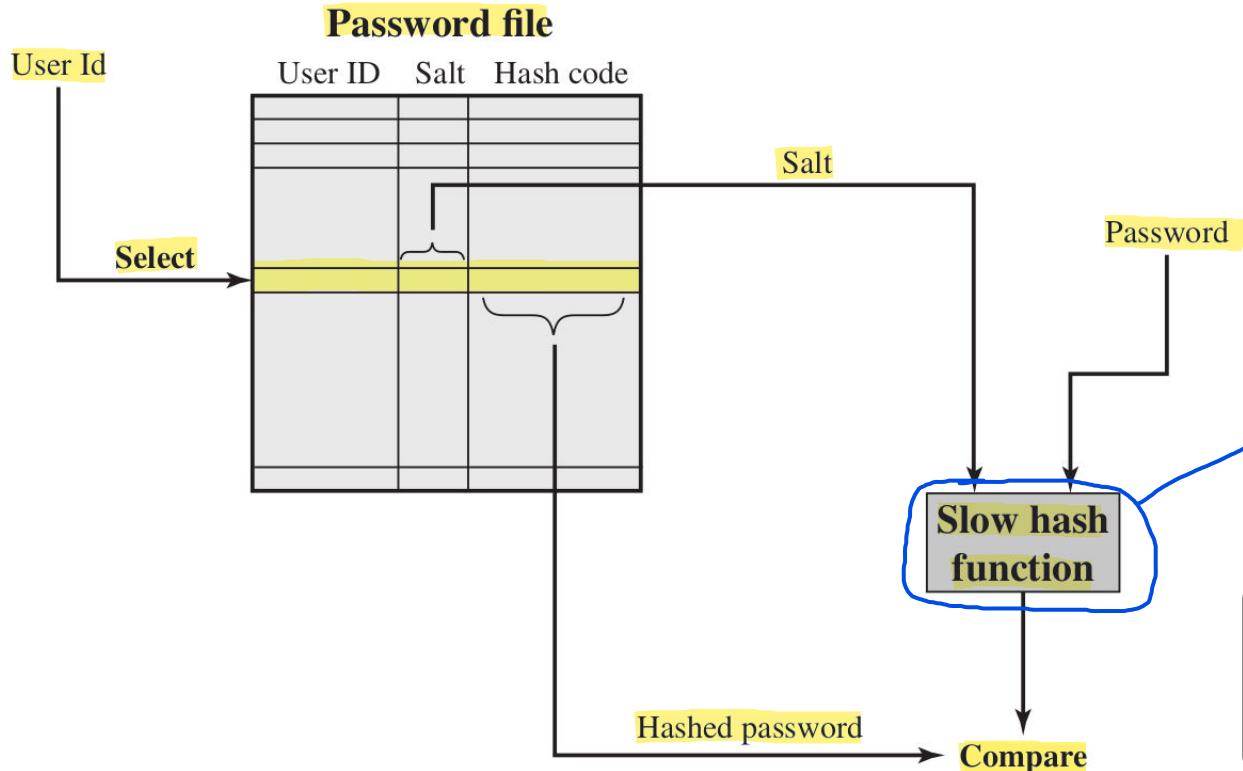


Loading a new password

Hashed and Salted Passwords



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Why does the “Verify” step have to be slow?

Because of a fundamental mathematical rule of hashing: the algorithm used must be exactly the same. If you generated the hash in the database using a slow hash function, when the user attempts to log in, the server must repeat exactly the same steps to verify if the result matches.

This introduces a devastating defensive asymmetry:

- > For the legitimate user (1 attempt): Waiting 100 ms or 200 ms to log in to the site is perfectly acceptable. The user experience remains excellent.
- > For the online attacker, it further slows down their network-based attempts, exhausting their resources.
- > For the offline attacker, if the algorithm requires a lot of CPU and memory, the attacker cannot use super-fast graphics cards (GPUs) or dedicated chips (ASICs) to parallelize the attack locally, because the “slowness” of the function requires hardware resources that GPUs cannot easily scale.

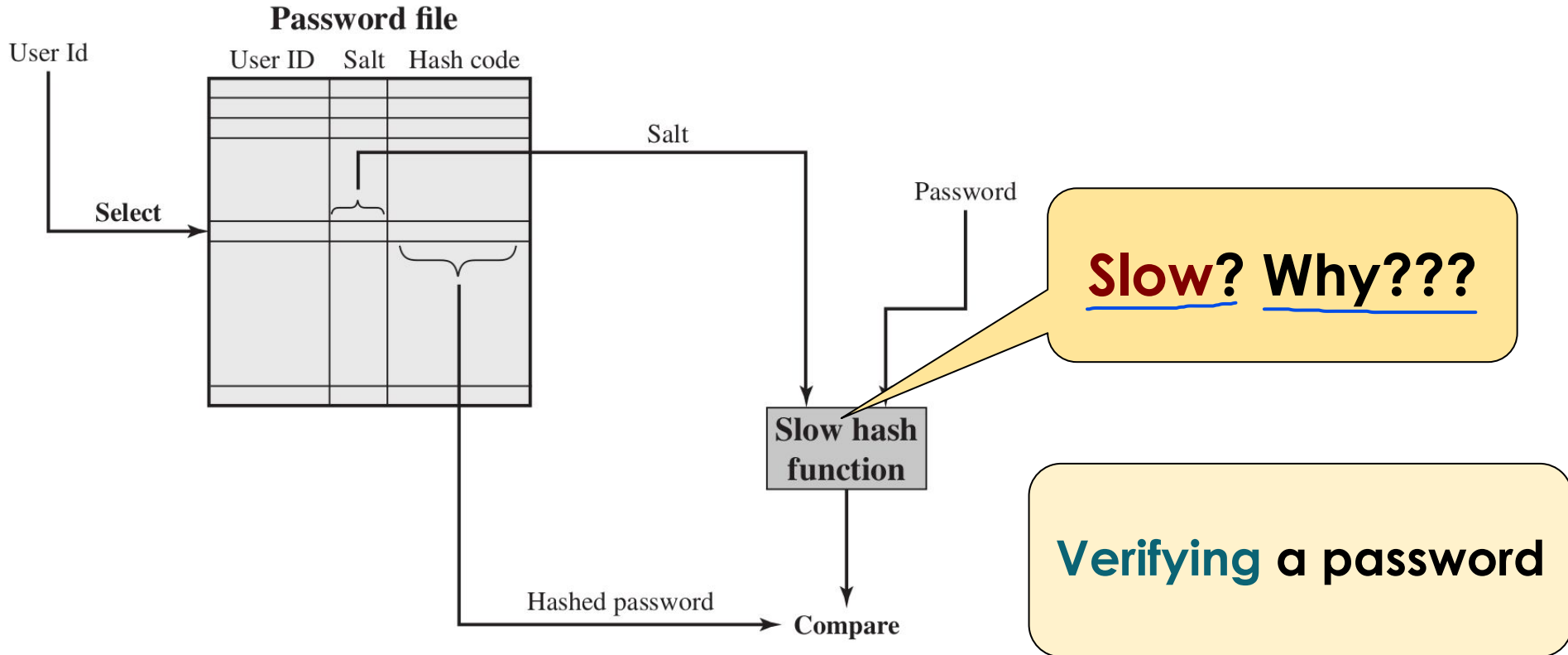
When logging in, the user sends their UserID and password to the system. The system retrieves the user's specific salt value from the database, computes $\text{Hash}(\text{password} + \text{salt})$, and checks whether it matches the stored hash to verify the user's identity.

Verifying a password

Hashed and Salted Passwords



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Hashed Passwords Mechanism



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Thanks to the salt, if two users choose the exact same password, their hashes in the database will be completely different. A system administrator or hacker looking at the file will never see two identical entries and will not be able to tell who has the same password.

- Prevents **duplicate passwords** from being **visible** in the **password file**
- Greatly **increases the difficulty** of **offline dictionary attacks**:
 - for a **salt** of length **b bits**, the **number of possible passwords is increased by a factor of 2^b**
- Becomes nearly impossible to find out whether a person with **passwords on two or more systems has used the same password** on all of them

Hashing

This is not clear! Why???

Salting



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Prevents **duplicate passwords** from being **visible** in the **password file**
- Greatly **increases the difficulty** of **offline dictionary attacks**:
 - for a **salt** of length **b bits**, the **number of possible passwords is increased by a factor of 2^b**
- Becomes nearly impossible to find out whether a person with **passwords on two or more systems has used the same password** on all of them

Hashing

This is not clear! Why???

Salting

- Prevention of Brute-Force Attacks (No Pre-computation): Without a salt, the attacker calculates the hash of a dictionary password only once and compares it simultaneously against all users in the database. With a salt, since each user has a unique, random salt, the final hash is different for everyone. The attacker is forced to recalculate the entire dictionary from scratch for every single user in the database.

- Rainbow Tables Are Useless: Rainbow Tables are massive databases of pre-computed hashes used by hackers to find passwords in a matter of milliseconds. The salt introduces 2^b possible variants (where b is the bit length of the salt). To still use Rainbow Tables, the attacker would have to generate a table containing every password combined with all 2^b salt variants. This requires storage space (Petabytes/Exabytes) that is mathematically and commercially impossible to manage.

- Prevents **duplicate passwords** from being **visible** in the **password file**
- Greatly **increases the difficulty** of **offline dictionary attacks**:
 - for a **salt** of length **b bits**, the **number of possible passwords is increased by a factor of 2^b**
- Becomes nearly impossible to find out **passwords on two or more systems** has **of them**

To prevent the use of
rainbow tables (to be
discussed later)

Unix (very old) Implementation



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Original scheme**

The system accepted passwords up to 8 characters long. If a user entered a longer password (e.g., password123), the system ignored everything after the eighth character, treating only "password" as the password. This greatly limited the complexity (the "key space").

- the password is **up to eight printable characters** in length
- **12-bit salt** used to modify **DES encryption** into a **one-way hash**

- **function**

→ At the time, there were no dedicated hash functions like SHA. Engineers took DES (a symmetric encryption algorithm) and modified it to function as a one-way hash function. To do this, they used a salt of only 12 bits. This salt altered the internal workings of DES, ensuring that the standard algorithm itself could not be used to decrypt the data.

- **zero value** repeatedly **encrypted 25 times**
- output translated to **11 character sequence**

→ This was the actual hashing process. The system took an empty string (a sequence of zeros), used the user's password as an "encryption key" (modified by the salt), and encrypted these zeros 25 times in a row. This 25-time loop was intended to intentionally slow down the computation to counter the hardware of the time (it was a primitive Slow Hash Function).

→ The final result of this process was converted into an 11-character readable string and saved in the password file.

- **Now** regarded as **inadequate (i.e., really insecure)** Because DES is insecure, salt and hash code is too small
- **Sometimes** (less and less) **still required for compatibility** with existing account management software or multi-vendor environments

Improved Implementation



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Recommended hash function was based on **MD5** (now it is **SHA512**)
 - **salt** of up to **48-bits** (or sometimes even more)
 - **password length** is **unlimited**
 - produces **128-bit** hash (MD5), SHA-512 produces 512-bit
 - uses an **inner loop** with **1000 iterations** to achieve **slowdown** →

As we saw earlier with Slow Hash Functions, the system does not calculate the hash just once. It runs an internal loop, repeating the calculation 1,000 times (or many more in modern systems). This introduces an intentional delay that drastically slows down hackers' offline brute-force attacks.

-  OpenBSD is a free, open-source, Unix-like operating system
OpenBSD uses Blowfish block cipher based hash algorithm called

Bcrypt

- **most (?) secure version of Unix hash/salt scheme** →
- **uses 128-bit salt** to create 192-bit hash value

The main reason is that Bcrypt has an adaptive cost factor. This means that the administrator can decide how many computational rounds the algorithm should perform (e.g., 2^{10} , 2^{12} , etc.). As computers become faster over the years, simply increasing this number is enough to keep the algorithm slow and secure, without having to change the software.

Improved Implementation



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Recommended hash function was based on **MD5** (now it is **SHA512**)
 - **salt** of up to **48-bits** (or sometimes even more)
 - **password length** is **unlimited**
 - produces 1
 - uses an inner **slowdown**
- **OpenBSD** uses **Bcrypt** algorithm called Bcrypt
 - most (?) secure version of Unix hash/salt scheme
 - uses 128-bit salt to create 192-bit hash value

Broken!!!!

Improved Implementation



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Recommended hash function was based on **MD5** (now it is **SHA512**)
 - **salt** of up to **48-bits** (or sometimes even more)
 - **password length** is **unlimited**
 - produces **128-bit** hash
 - uses an **inner loop** with **1000 iterations** to achieve **slowdown**
- **OpenBSD** uses Blowfish block cipher based hash algorithm called Bcrypt
 - most (?) secure version of Unix hash/salt scheme
 - uses 128-bit salt to create 192-bit hash value

How do you defend against password cracking? By using a salt, which fights password cracking in two ways: It renders rainbow tables difficult to use and it multiplies the attacker's workload. If two users have the exact same password (e.g., Milano2026), they will have two different salts and therefore two completely different hashes in the database. The attacker is forced to perform a separate dictionary attack for each user in the system, drastically slowing down their attempt at a mass breach.

Password Cracking

an attacking process used to recover a plaintext password from its hash (the mathematical fingerprint of the password).



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

1 • Dictionary attacks

- develop a **large dictionary of possible passwords** and **try each of them against the password file**
- each password must be **hashed using each salt value** and then **compared to stored hash values** (number of hashes: dictionary size * number of users; if we use salt: hashes that must be computationally calculated during attack)

2 • Rainbow table attacks

- **pre-compute tables of hash values for all salts** → The attacker's ideal goal would be to pre-compute all possible hashes for all passwords by combining them with every existing salt, so that they would only need to perform a quick search in their file at the time of the attack.
- a **mammoth table** of hash values
- can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length** → The longer the salt, the more the 2^b factor (the total number of salt combinations) explodes mathematically, making the creation of this "mammoth table" physically impossible due to a lack of storage space and computational time.

Password Cracking



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Dictionary attacks**

- develop a **large dictionary of possible passwords** and **try each of them against the password file**
- each password must be **hashed using each salt value** and then **compared to stored hash values**

- **Rainbow table attacks**

- **pre-compute tables** of hash values **f**
- a **mammoth table** of hash values
- can be **countered** by using **large hash length**

The salt values are stored (in
clear) in the password file

Password Cracking



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The introduction of a salt destroys rainbow table attacks for three key reasons:

- A. It eliminates the universality of the table: The strength of a rainbow table lies in its universality, a hacker calculates it once and can use it against any database in the world, because the password "123456" will always produce the same hash. With a salt, if two users have the same password "123456," their hashes will be completely different due to their unique salts. The hacker's standard table becomes useless.
- B. It causes a combinatorial explosion of space: To bypass the salt, the hacker would theoretically have to calculate a different rainbow table for every possible salt value. With a 128-bit Salt, there are 2^{128} possible Salt combinations. To store the Rainbow Tables for all these combinations would require such immense storage space that it would take more hard drives than there are atoms in the observable universe. Consequently, pre-computing the table becomes mathematically and physically impossible.
- C. It forces the attacker to slow down: Without the ability to use pre-computed tables, the time-memory trade-off breaks down. The hacker loses their advantage and is forced to revert to the old brute-force or real-time dictionary attack, calculating hashes one at a time for every single user in the stolen database. If the database also uses slow hashing functions, the attack becomes so slow that it is inefficient.

- **Dictionary attacks**

- develop a **large dictionary of possible passwords** and **try each of them against the password file**
- each password must be **hashed using each salt value** and then **compared to stored hash values**

- **Rainbow table attacks**

- **pre-compute tables** of **hash values for all salts**
- a **mammoth table** of hash values
- can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

Password Cracking



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Dictionary attacks**

- decompile the binary and **try each of them** against the hashes
- each password is hashed with the same salt value and then **compared to** the stored hashes

The goal is to boost the efficiency of the attack

- **Rainbow table attacks**

- **pre-compute** tables of hash values for all salts
- a **mammoth table** of hash values
- can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

Password Cracking



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Dictionary attacks**

- develop a **large dictionary of possible passwords** and **try each of them against the password file**
- each password is **hashed** and then **compared to stored hash values**

It is going to be huge!

- **Rainbow table attack**

- **pre-computed tables** of hash values for all salts
- a **mammoth table** of hash values
- can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

Password Cracking



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Dictionary attacks**

- develop a **large dictionary of possible**
the password file
- each password must be **hashed using**
stored hash values

Really too big to be stored

- **Rainbow table attacks**

- **pre-compute tables** of hash values for all sa
- a **mammoth table** of hash values
- can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

Password Cracking

A Rainbow Table is a cryptographic hacking tool designed as a Time-Memory Trade-Off (TMTO). It bridges the gap between two inefficient extremes; pure brute-force attacks (which require unsustainable computing time) and flat lookup databases (which require impossible amounts of storage space). Its massive efficiency in recovering plaintext passwords from stolen hashes relies on three main architectural principles:

- 1. Shifting the Computational Burden (Pre-computation)

> The Concept: The computationally expensive work of calculating billions of mathematical hashes is entirely shifted to the pre-attack phase.

> The Benefit: Real-time computation during the actual breach is eliminated. Finding a password is reduced to a simple database query, turning a process that would normally take days or weeks into a virtually instantaneous lookup.

- 2. Cryptographic Chain Compression (The Space Saver)

> The Concept: Storing a flat database of every single password-hash pair would require petabytes of storage. Rainbow Tables solve this by grouping passwords into rigid mathematical "chains" using alternating Hash Functions and Reduction Functions.

> The Benefit: On the hard drive, the attacker only stores the first password and the final hash of each chain. If a chain is 1,000 steps long, 99.9% of the data is compressed into thin air, shrinking the total file size from Terabytes to a few Gigabytes while retaining the ability to reconstruct intermediate passwords on the fly.

- 3. Target-Independent Reusability

> The Concept: A Rainbow Table is mathematically bound only to a specific cryptographic algorithm (e.g., MD5 or SHA-256) and a defined character set, rather than a specific target system.

> The Benefit: Once a table is generated, it becomes a permanent, highly reusable asset. The attacker can deploy the exact same "mammoth table" against any system or database in the world that utilizes that specific unsalted hashing function.

● Dictionary attacks

- develop a list of words
- the password
- each password
- stored hash value

The goal is to boost the
efficiency of the attack

try each of them against

and then compared to

● Rainbow table attacks

- pre-compute tables of hash values
- a mammoth table of hash values
- can be countered by using a sufficiently large salt value and a sufficiently

Can you explain
HOW and **WHY?**

large hash length

Reduction Chains: Instead of storing every single explicit Password -> Hash pair, the table generates a sequence of alternating hashes and passwords using a Reduction Function (R). Note: The reduction function does not invert the cryptographic hash; it simply maps an output hash value back into a new, valid plain-text string (a new password guess). Space Optimization: For a chain spanning thousands of iterations, the database discards all intermediate values and stores only the first password (Start of Chain) and the final hash (End of Chain) as a single row on the disk. When the attack occurs, simply search for the stolen hash in the "End of Chain" column, and if there is a match, reconstruct that specific chain in a matter of seconds to find the plaintext password.

Password File Access Control



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

We can block some offline guessing attacks by denying access to encrypted passwords

Countermeasures

1

make available only to
privileged users

2

shadow password file:
a separate file from the
user IDs where the
hashed passwords are
kept

Privilege escalation
by OS vulnerability

weakness in
the OS that
allows
access to the
file

Even if you implement these countermeasures, the system is not 100% secure. The slide lists five vulnerabilities that could still allow an attacker to bypass the defenses or steal the file:

vulnerabilities

accidentally changes the
permissions of the shadow file
accident with
permissions
making it
readable

users with
same
password
on other
systems

access
from
backup
media

sniff
passwords
in network
traffic

Password Selection Strategies



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

user education

computer generated passwords

reactive password checking

proactive password checking

Password Selection Strategies



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

user education

computer generated passwords

reactive password checking

proactive password checking

users can be told the
importance of using
hard to guess
passwords
and can be provided
with guidelines for
selecting strong
passwords

Password Selection Strategies



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

user education

computer generated passwords

reactive password checking

proactive password checking

hard to guess for
attackers but users
have trouble
remembering them

Password Selection Strategies

Computer-generated passwords stored in password managers eliminate the risks associated with the inherent weakness of human-generated passwords (by preventing dictionary attacks and attacks targeting popular passwords). However, this strategy shifts the threat model: the focus of security shifts from protecting individual accounts to protecting the master password and the application itself, which becomes the single point of failure for the entire system.

user education

computer generated passwords

reactive password checking

proactive password checking

a (good) possibility is
to use password
managers: with some
attention to the
specific threat model

Password Selection Strategies



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

user education

computer generated passwords

reactive password checking

proactive password checking

the system
periodically runs its
own password
cracker
to find guessable
passwords

Password Selection Strategies



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

user education

computer generated passwords

reactive password checking

proactive password checking

**user is allowed to
select their own
password, however
the system checks to
see if the password is
allowable, and if not,
rejects it**

Password Selection Strategies



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

user education

computer generated passwords

reactive password checking

proactive password checking

**the goal is to
eliminate guessable
passwords while
allowing the user to
select a password that
is memorable**

In contrast to reactive countermeasures (which are implemented after an account has been compromised), the proactive approach takes action at the very moment a user is creating or changing their password, preventing them from choosing a weak one.

Proactive Password Checking



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

three levels of increasing complexity

- 1 • **Rule enforcement** The system rejects the password if it does not meet certain quantitative requirements.
 - **specific rules** that passwords **must adhere to**
 - 2 • **Password cracker** Instead of simply applying mathematical rules, the server acts as an attacker to test the strength of the password in real time.
 - compile a **large dictionary** of passwords **not to use** (+ **variations**)
 - 3 • **Other more sophisticated mechanisms** They represent the state of the art in modern security, going beyond the limitations of simple static dictionaries.
 - **neural networks-based evaluation** The neural network estimates the actual entropy and predictability of the structure chosen by the user, determining whether an automated brute-force program would take just a few seconds or years to guess it.
 - **password invalidation** based on **public data breaches** The system connects in real time (via secure, anonymized APIs) to global databases that collect all the passwords stolen worldwide in past corporate hacking attacks.
- (e.g., “[Have I Been Pwned](#)” + [API](#))

Password Cracking using AI?



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

ars TECHNICA

BIZ & IT TECH SCIENCE POLICY CARS GAMING & CULTURE STORE FORUMS SUBSCRIBE

Q ≡ SIGN IN

WANT CHEESE WITH THAT NOTHINGBURGER? —

Meet PassGAN, the supposedly “terrifying” AI password cracker that’s mostly hype

AI cracking is on par with conventional methods, but you’d be forgiven for thinking otherwise.

DAN GOODIN - 4/12/2023, 6:22 PM



By processing millions of data points, AI independently learns the probabilistic distribution of human passwords. It figures out on its own which combinations of letters, numbers, and symbols are most likely to be chosen by a real person, mapping the “psychology” behind password creation.

«These **generative adversarial networks**

generate password guesses after

autonomously learning the distribution

of passwords by processing the spoils of

data collected from

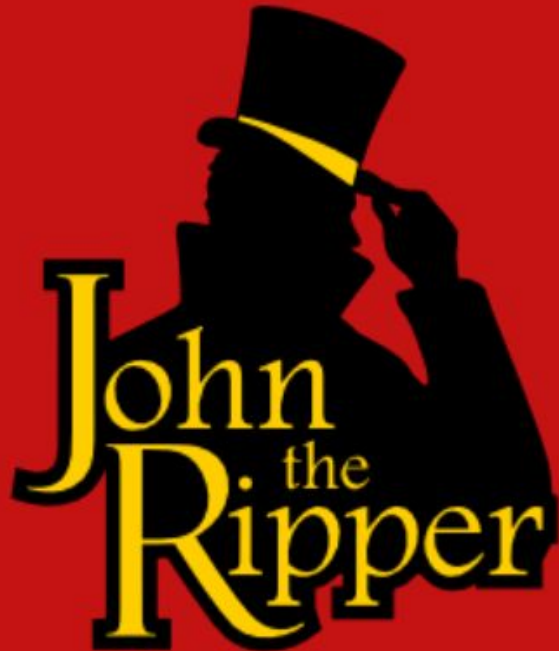
previous real-world breaches»

Hackers train them using datasets from massive historical data breaches that actually occurred on the web. By studying billions of real passwords chosen by real people, the AI becomes an absolute expert at predicting human patterns.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Password Cracking



**Offline password cracking
using John the Ripper (JtR)**

How does it work?

Check it out!

T6



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Tokens

Until now, you have explored authentication based on "something you know" (passwords). This slide introduces the second category: authentication based on "something you have" (token-based authentication).

Token-based Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Specifically, the table analyzes the technological evolution of the various types of cards used as physical authentication tokens. The evolution is presented from top to bottom, starting with the oldest, purely mechanical technologies and progressing to the most modern and intelligent ones.

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card

Types of cards used as Tokens

These cards feature raised alphanumeric characters only on the front side of the card. This is the most basic level of technology. There are no electronic or computer components inside the card. Authentication or transactions took place purely mechanically: the card was inserted into a manual device which, by applying pressure to carbon paper, copied the raised numbers onto the receipt. This technology is now obsolete in terms of cybersecurity.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Token-based Authentication

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card



They have a black magnetic strip on the back and printed characters (usually flat or raised (embossed)) on the front. The magnetic strip on the back contains data stored in the form of aligned magnetic particles. When the card is swiped through the reader, the reader reads the stored static data (e.g., account number and cardholder name). The security risk: The data on the magnetic strip is static and not protected by encryption. Anyone with an inexpensive device (a skimmer) can easily read and duplicate (clone) the entire magnetic strip onto a blank card.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Token-based Authentication

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card



They contain a real integrated electronic memory circuit inside. Unlike magnetic stripes, the data is stored on a memory chip. The reader can read the data and even modify it (for example, by deducting the remaining credit from a phone card). Although this is a step forward, these cards contain only hardware memory but lack a processor. Security risk: This means they cannot perform calculations or cryptographic algorithms. Data is simply read or written passively, making them vulnerable to advanced manipulation.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Token-based Authentication

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card

Obsolete!

Memory Cards

OBSOLETE

They have been phased out in favor of Smart Cards due to critical architectural vulnerabilities. Here is the detailed breakdown of why they are obsolete:

1. Lack of Processing Power
2. Vulnerability to Cloning (Skimming): the information inside them is almost always stored in clear text.
3. Susceptibility to Replay Attacks
4. Mechanical and Physical Wear: The magnetic particles wear out over time, degrade when exposed to external magnetic fields (demagnetization), and get scratched easily. Modern contactless smart cards (NFC/RFID) remove mechanical friction entirely, lasting much longer and offering a better user experience.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Can **store** but **do not process data**
- The most common^{type} is the **magnetic stripe card**
- Can include an **internal electronic memory**
- Can be used alone for **physical access** (e.g., hotel rooms)
- Provides significantly greater security when **combined** with a **password** or **PIN** (e.g., ATMs)
- **Drawbacks** of memory cards include:
 - requires a **special reader**
 - **loss of token** Since it is a physical object, the user may lose it, have it stolen, or damage it. If the user loses the token, they will be locked out of the system until a new one is issued.
 - information are often **stored in clear**

Since the card cannot perform encryption, data is stored on the magnetic stripe or in the card's memory in plain text. Anyone with a compatible reader costing just a few euros (a skimmer) can intercept this data, read it, and copy it onto a blank card, thereby carrying out a cloning attack.

Memory Cards



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Can **store** but **do not process data**
- The most common is the **magnetic stripe card**
- Can include an **internal electronic memory**
- Can be used alone for **physical access** (e.g., *hotel rooms*)
- Provides significantly greater security when **combined** with a

Is the **locker** properly
reprogrammed at **each**
room change?

Memory Cards



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Can **store** but **do not process data**
- The most common is the **magnetic stripe card**
- Can include an **internal electronic memory**
- Can be used alone for **physical access** (e.g., hotel rooms)
- Provides significantly greater security when **combined** with a

When the card is used on its own (single-factor authentication: something you have). A classic example is a hotel key card. If you find one on the ground, you can use it to enter the room, because the system only checks for the presence of the card, not who is holding it. Security in this case is low to moderate.

Is the **locker** properly
reprogrammed at each

every time there is
a new guess in the room
room change?

NO! So everybody that
have access of the
memory card can
create a copy of it

Sweet dreams :-)

They contain both electronic memory and an internal microprocessor. They are, in effect, tiny computers encapsulated in a piece of plastic. They can store cryptographic keys and execute authentication algorithms directly on the card itself, without ever exposing sensitive data to the outside world.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Token-based Authentication

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart	Electronic memory and processor inside	Biometric ID card
Contact	Electrical contacts exposed on surface	
Contactless	Radio antenna embedded inside	

- Contact: These cards have gold-plated electrical contacts visible on the surface (electrical contacts exposed). The card must be physically inserted into the reader.



Smartcards

Due modi in cui una Smartcard può interagire con il mondo esterno o con l'utente:

- Manual interfaces: Alcune smartcard speciali includono sulla loro superficie fisica un minuscolo tastierino (keypad) e un piccolo display a cristalli liquidi. L'utente interagisce direttamente con la carta (ad esempio digitando un PIN sul tastierino della tessera stessa) e la carta mostra un codice sul display.
- Electronic interfaces: La smartcard comunica elettronicamente inviando e ricevendo dati digitali da un lettore/scrittore compatibile. Come abbiamo visto nella primissima tabella, questa interfaccia elettronica può essere a contatto (inserendo la tessera nel lettore che tocca i contatti dorati) o contactless (tramite un'antenna radio NFC/RFID integrata).



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Physical characteristics:** include an **embedded microprocessor**
- **Interface:**
 - **manual interfaces** include a keypad and display for interaction
 - **electronic interfaces** communicate with a compatible reader/writer
- **Authentication protocol:**
 - **static** → The simplest (and least secure) protocol. The card contains a fixed piece of data (e.g., a certificate or a key) and sends it to the reader. Even though the microprocessor protects the data from immediate cloning, the output sent is always the same.
 - **dynamic password generator** → The smart card functions as an OTP (One-Time Password) generator. Using an internal algorithm based on time or an event counter, the microprocessor generates a one-time password that changes continuously (e.g., every 30 seconds). Even if a hacker intercepts the current password, they won't be able to reuse it because it will expire immediately.
 - **challenge-response** → It is the most secure and advanced protocol. When you insert the smart card, the server sends a challenge to the card—a random number generated on the spot. The smart card's microprocessor takes this number, combines it with the secret key stored inside, and calculates an encrypted response. The response is sent back to the server, which verifies the correctness of the calculation. Why is it exceptional? The secret key never travels over the network and is never revealed to the reader. Furthermore, since the challenge changes with every single login, intercepting the data is completely useless to a hacker (total protection against replay attacks).

Smartcards



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Physical characteristics:** include an **embedded microprocessor**
- **Interface**
 - **manu** Not nice for security ay for interaction
 - **electronic** communicate with a compatible reader/writer
- **Authenticati** protocol:
 - **static**
 - **dynamic** password generator
 - **challenge-response**

Smartcards



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Physical characteristics:** include an **embedded microprocessor**
- **Interface:**
 - manual interaction for interaction
 - electronic interaction with compatible reader/writer
- **Authentication protocols:**
 - static
 - **dynamic password generator**
 - challenge-response

Often, it requires
synchronization

Smartcards



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

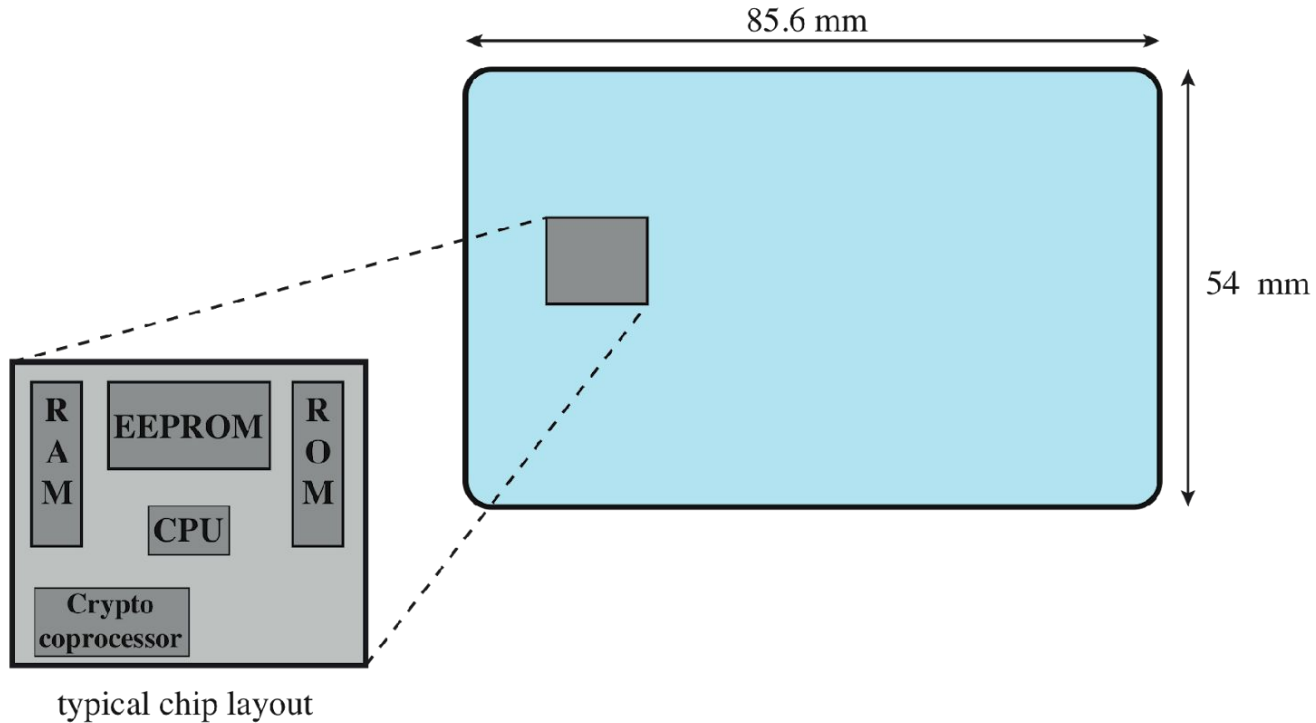
- **Physical characteristics**: include an **embedded microprocessor**
- **Interface**:
 - **manual interfaces** include a keypad and display for interaction
 - **electronic interfaces** include a reader/writer
- **Authentication** protocols
 - **static**
 - **dynamic** password generator
 - **challenge-response**

To be discussed later ...

Smartcards



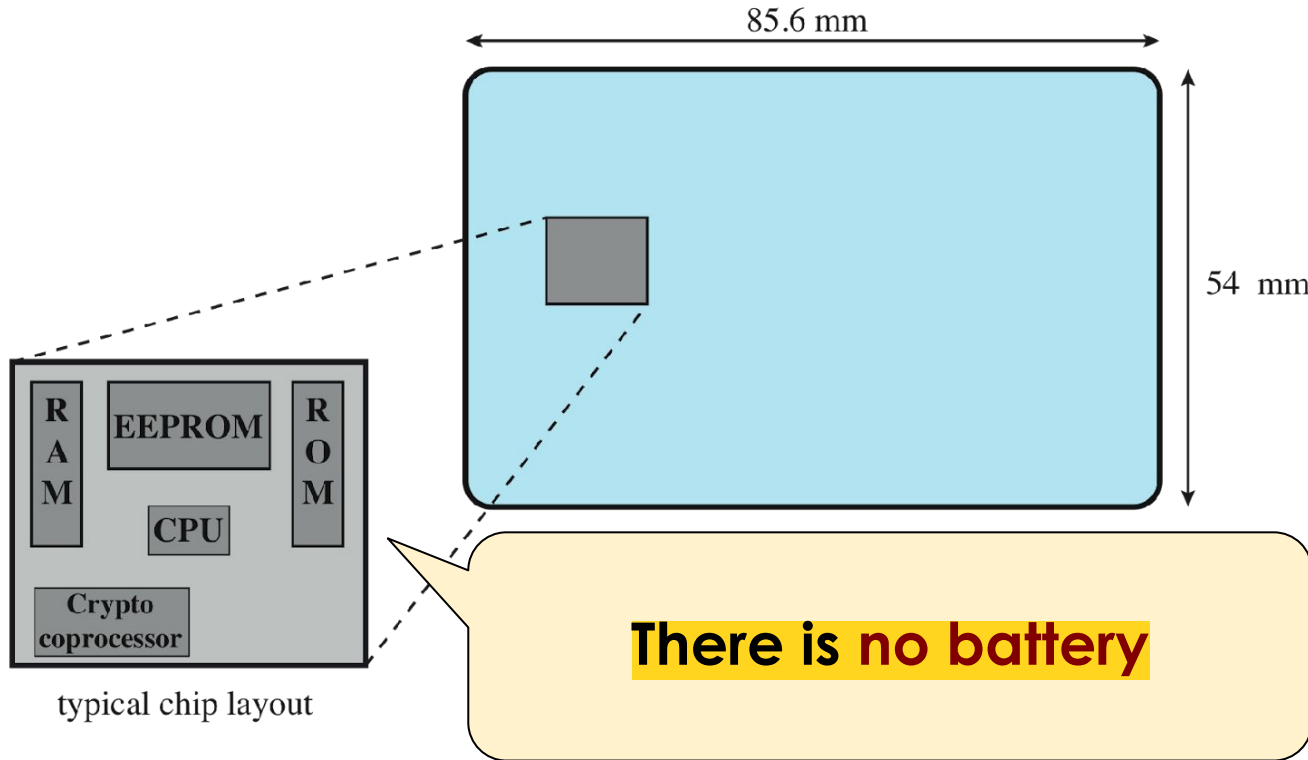
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Smartcards



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Token-based Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Card Type	Description
Embossed	Raised characters on front
Magnetic stripe	Magnetic bar on back
Memory	Electronic memory in chip
Smart Contact	Electronic memory and electrical contacts on front
Contactless	Radio antenna embedded in the card

- Contactless: These cards have no visible contacts. A tiny radio antenna is embedded inside the plastic. They communicate over very short distances using NFC or RFID technology simply by holding the card near the reader.



Token-based Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**No major vulnerabilities
have been reported**

Card	
Embossed	
Magnetic strip	
Memory	Electronic memory in
Smart Contact	Electronic memory and Electrical contacts e
Contactless	Radio antenna emb



Token-based Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

No major vulnerabilities
have been reported

Card	
Embossed	
Magnetic strip	
Memory	Electronic memory in
Smart Contact	Electronic memory a Electrical contacts e
Contactless	Radio antenna emb

For small value
transactions, the PIN is
not required



In the world of security, there's a rule of experience: the more secure a system is, the less convenient it is, the less secure it is. Contactless payment is the perfect example. Maximum Usability (Convenience): To pay for your coffee at the café, all you have to do is hold your card or phone up to the reader for a fraction of a second. You don't have to insert the card into the reader, you don't have to enter a PIN, and you don't have to sign anything. It's fast and frictionless. The Security Risk: If you lose your card or a thief steals it, they too can go to the café and pay for a coffee simply by holding it up, because the underlying system doesn't verify who is holding the card. Furthermore, theoretically, an attacker with a portable POS reader hidden in a backpack could approach you in a crowded subway and skim money from your jacket (a proximity skimming attack). To maintain high usability without compromising security, designers had to find a compromise (a trade-off): for example, allowing contactless payments without a PIN only below a certain monetary threshold (e.g., up to €50).



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Token-based Authentication

- There is a trade-off between usability and security
- Security is partially based on “usage pattern analysis”

When the Usage Pattern Analysis algorithms detect an anomaly compared to your historical profile, the system reacts immediately as a security measure: it either rejects the contactless transaction, forcing the cardholder to insert the card and enter the PIN (which the thief does not know), or temporarily blocks the card by sending an SMS or app notification to the actual owner.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

NON RIENTRA NEL PROGRAMMA D'ESAME

Remote Authentication

Remote (User) Authentication



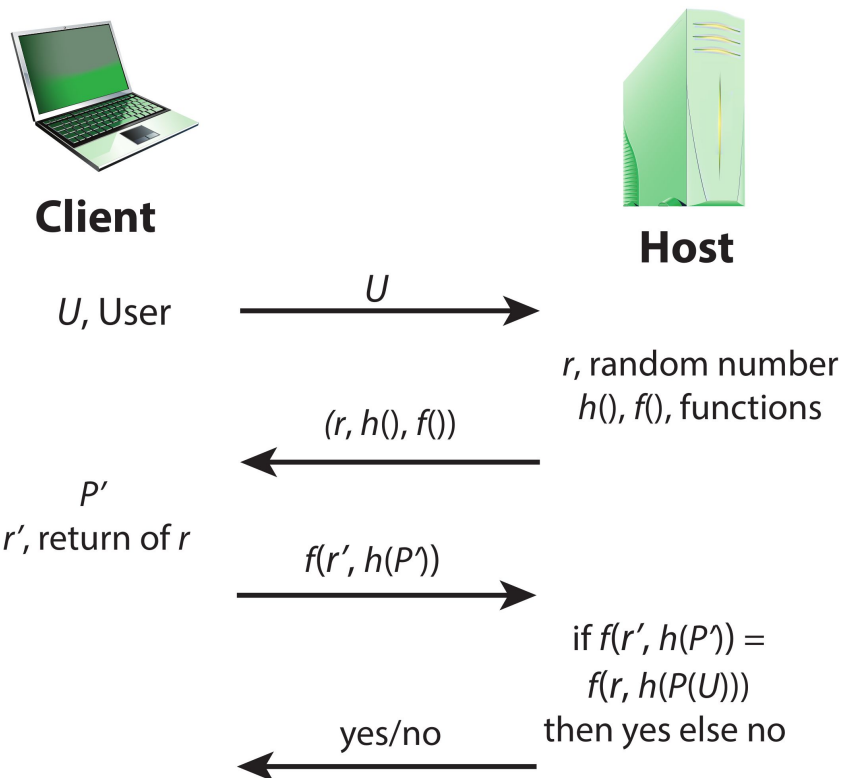
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Authentication** over a **network**, the Internet, or a communications link is **more complex** (vs. local)
- **Additional security threats** such as:
 - **eavesdropping, capturing** a password, **replaying** an authentication sequence that has been observed (and recorded by an attacker)
- Generally relies on some form of a **challenge-response protocol** to counter threats

Remote (User) Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Remote (User) Authentication

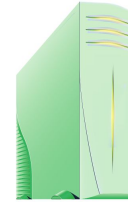


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Client

U , User



Host

r , random number
 $h()$, $f()$, functions

$(r, h(), f())$

$f(r', h(P'))$

if $f(r', h(P')) =$
 $f(r, h(P(U)))$
then yes else no

yes/no

P'
 r' , return of r

$r' = r$

password

nonce



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

NON RIENTRA NEL PROGRAMMA D'ESAME

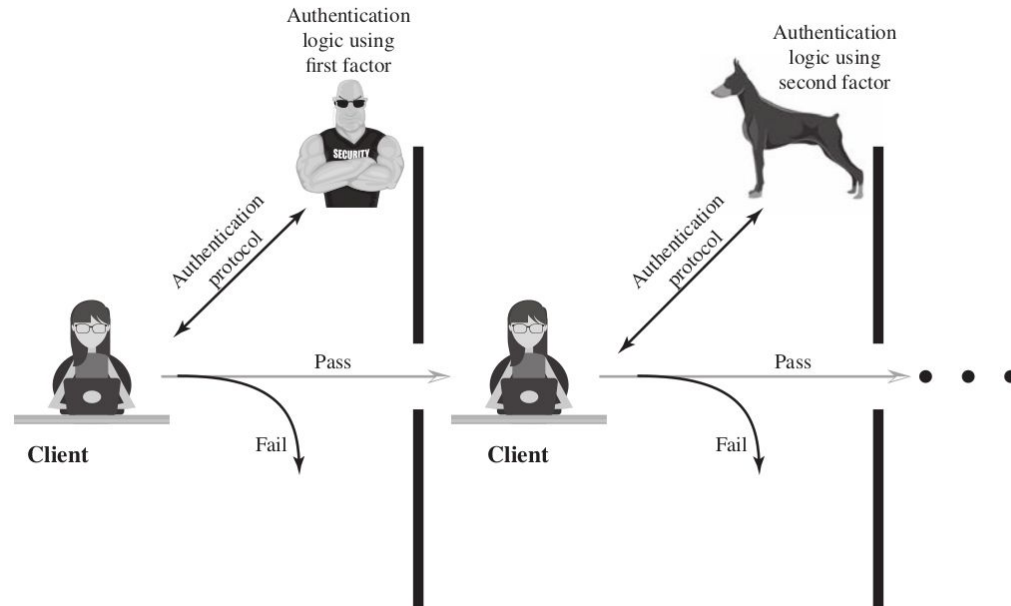
Multi-Factor **Authentication**

Multi-factor Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication means**
- Implementations that use **multi-factors** are considered to be **stronger** than those that use only one factor



Multi-factor Authentication



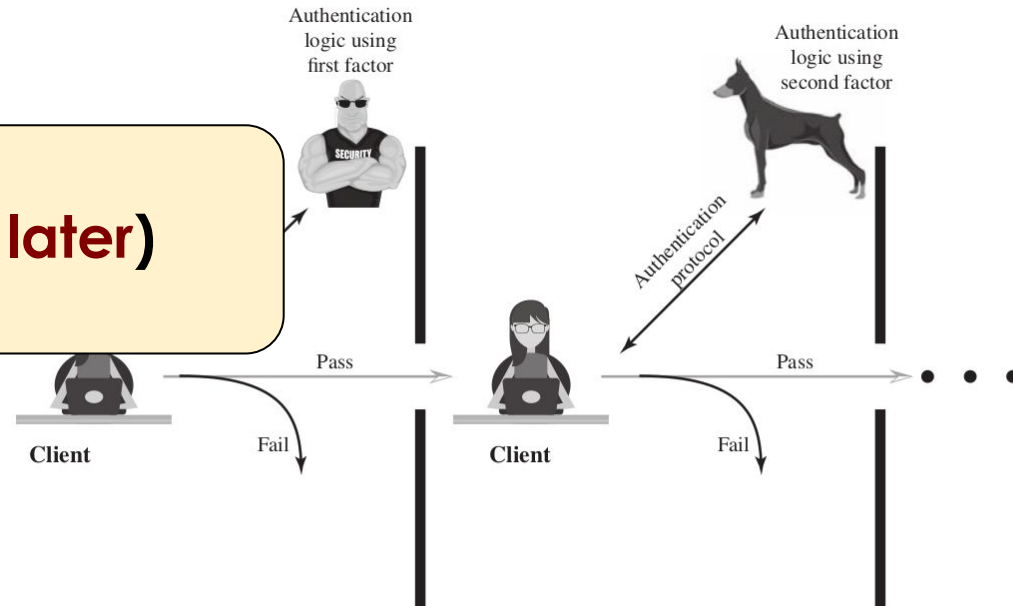
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication**

means

Yes but ... (see later)

- Implementation of **multi-factor** is considered to be **stronger** than those that use only one factor



Example: two-step authentication



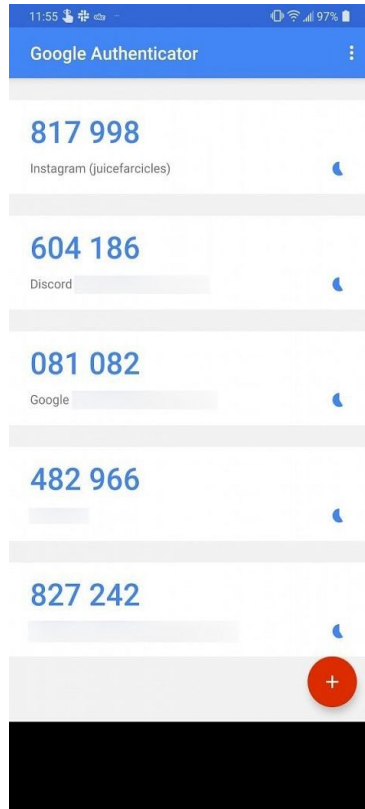
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Two** (or more) **steps** are used to **verify the identity** of an entity that is trying to access services
- **Possible implementation (both steps are mandatory):**
 - a. something that the **users knows** (e.g., username + password)
 - b. something that is **generated by a device owned by the user** (e.g., a dynamic PIN), or a **one-time code** delivered by **SMS**

Example: two-step authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Google Authenticator



Example of Multi-Factor Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

“Out of band verification
using **SMS** is **deprecated**,

and will no longer be

allowed in future releases of
this guidance” (NIST)

the **identity** of an entity that is

are mandatory):

username + password)

device **owned** by the user (e.g., a

dynamic PIN), or a **one-time code** delivered by **SMS**

Example of Multi-Factor Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

“Out of band verification
using **SMS** is **deprecated**,

and will no longer be
allowed in future releases of
this guidance” (NIST)

**SMS are not good for
security**

the identity of an entity that is

are mand

username (password)

device used by the user (e.g., a

dynamic PIN), or a **one-time code** delivered by **SMS**

Multi-factor Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than**

one of the **authentication**

means

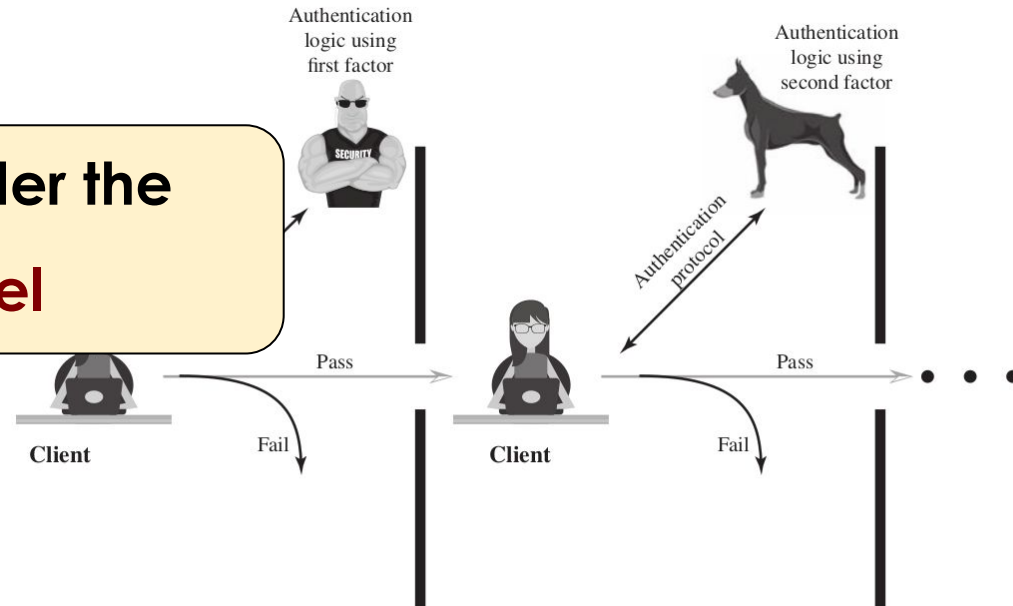
Carefully consider the
threat model

- Implementing

multi-factor is considered to

be **stronger** than those that

use only one factor



Multi-factor Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication**

means

Carefully consider the
threat model

- Implementations of multi-factor authentication are considered to be **stronger** than those that use only one factor



Authen
logic
first

Pa

Fail

For example: the
security of the
channel used for
the 2nd step
verification

Multi-factor Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication**

means

Carefully consider the
threat model

- Implementing multi-factor authentication should be **stronger** than those that use only one factor

Is it **IN band** or
OUT of band?

For example: the
security of the
channel used for
the 2nd step
verification

Multi-factor Authentication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication**

means

**Carefully consider the
threat model**

- Implementations of multi-factor authentication are considered to be **stronger** than those that use only one factor



Auth
log
fir

Fail

IN band: a single device (i.e. PC) is used to input both the factors... **what happens if that device (or its channel) is compromised?**



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

NON RIENTRA NEL PROGRAMMA D'ESAME

Passkeys

The next step... passkeys



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Home > Products > Google Identity > Authentication > Passkeys

Was this helpful?  

Passwordless login with passkeys



[Send feedback](#)



Passkeys: technical description



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The goal is to **completely replace** traditional **username** and **passwords**
- Passkeys are based on **public-key cryptography** (i.e., key pairs)
- Passkeys are stored on the **local storage of personal devices**, **not on remote servers**. This can reduce the impact of **server-side data breaches**
- **Phishing resistant**: each passkey is solely linked to the specific app or website it was created for

Passkeys: technical description



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

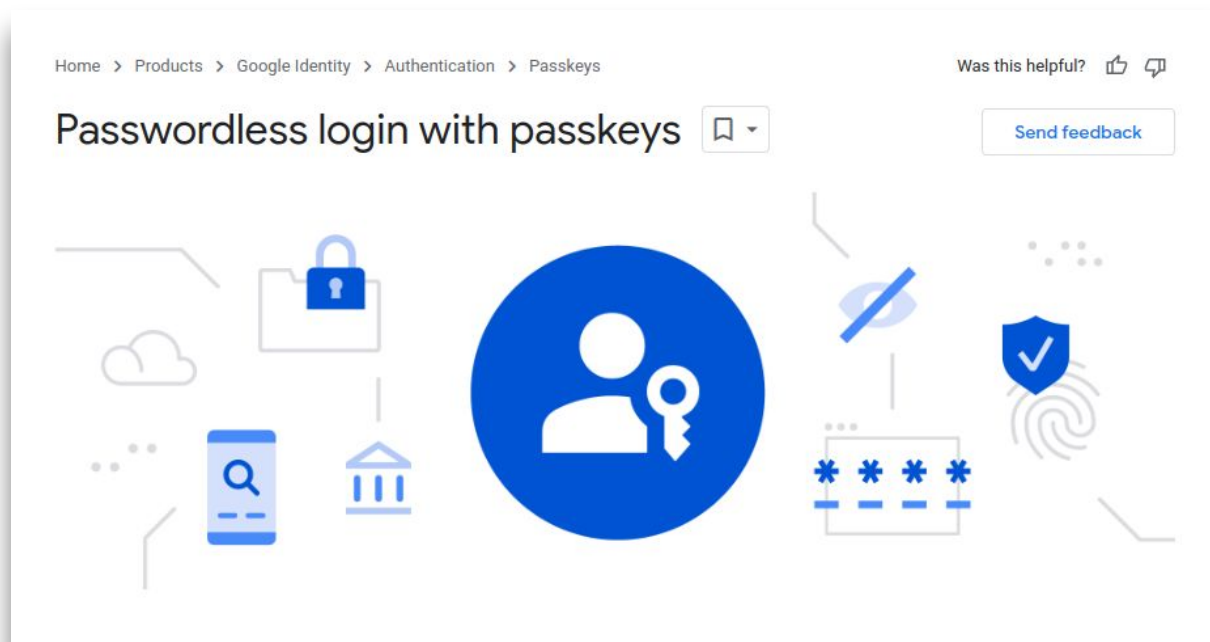
- Passkeys utilize **biometric verification** (i.e., fingerprint or facial recognition) for authorizing logins
- **Much better usability** than password
- Passkeys can be **synced between different devices** using the appropriate tools and considering the **effects on security** of this operation

The next step... passkeys



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Only partially supported and **deployed**
- Even if **standard**, there are relevant limitations in terms of **interoperability**
- What about the **threat model**?



Legal implications of biometric auth



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



- Authentication-based on **biometry** can have **different legal protection** than **passwords**
- What about the protection of **passkeys**?



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

Gabriele D'Angelo

<g.dangelo@unibo.it>

<http://www.cs.unibo.it/gdangelo/>

www.unibo.it

Attributions and Notes



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Most of the slides are from: “**Computer Security: Principles and Practice**”, 4th edition, W. Stallings and Lawrie Brown
- Some others are from: Dr. **Mehmet H. Gunes** (Department of Computer Science & Engineering - University of Nevada, Reno)
- Some others are from: **Mario Cagalj**, Ph.D. (Faculty of Electrical Engineering, Mechanical Engineering and Naval Architecture (FESB), University of Split)
- I wish to **thank all of them**, **any errors that remain are my sole responsibility**